
VASP Workflow

User Manual

Browser GUI · CLI Agents · SLURM HPC

Keivan Esfarjani

v1.6 · July 2026 | <https://github.com/KeivanS/VASP-Flow>

Contents

1	Introduction	2
1.1	Key features	2
1.2	Requirements	2
2	Installation	3
2.1	site.env — platform configuration	3
3	Browser GUI	3
3.1	Starting the GUI	3
3.2	Three-tab workflow	3
3.3	Setup tab — field guide	4
3.3.1	Structure and POTCAR	4
3.3.2	Functional	4
3.3.3	Spin / SOC	4
3.3.4	Other flags	4
3.4	Task checkboxes	4
3.5	Band structure options	5
3.6	DOS options	5
3.7	Workflow tab	5
3.8	Results tab	5
4	CLI Agents	5
4.1	vasp-agent.py — workstation	5
4.2	vasp-agent-slurm.py — SLURM clusters	5
4.3	Generated directory structure	6
5	High-Throughput Screening (Materials Project)	6
5.1	Execution order	7
6	instructions.txt Format	7
6.1	Example	7
6.2	Keyword reference	8

7	Methods	8
7.1	Functionals	8
7.2	Spin-orbit coupling	9
7.3	Magnetic materials (collinear spin)	9
7.4	GGA+U	9
8	Advanced INCAR Control	9
8.1	Raw INCAR passthrough	9
8.2	Constant-pressure relaxation	10
9	K-Path for Band Structure	10
9.1	Auto-detection (recommended)	10
9.2	Manual override	11
10	DOS Projections	11
11	Convergence Tests	11
12	Parallelization	12
12.1	Auto-defaults	12
12.2	Manual overrides	12
13	SLURM Cluster Settings	12
13.1	profiles/slurm.json	12
13.2	Per-project overrides (in instructions.txt)	13
14	Wannier90	13
15	Phonons (phonopy)	13
16	LOBSTER Bonding Analysis (COHP / COBI / COOP)	13
17	Worked Examples	14
17.1	MoS ₂ monolayer with SOC	14
17.2	Nickel: magnetism + GGA+U	14
17.3	GaAs: phonons with NAC	14
17.4	High-pressure relaxation with custom INCAR tags	15
17.5	Bi ₂ Se ₃ : topological insulator	15
18	Common OUTCAR Checks	16
19	Troubleshooting	16
20	Version History	16
	Addendum: High-Throughput Quick Reference	17

1 Introduction

VASP Workflow is a framework for setting up, running, and analysing DFT calculations with VASP. Two interfaces are provided:

- **Browser GUI** (`vasp-gui.py`) — a Flask web application: configure calculations through a form, run steps from the browser, view band-structure and DOS plots in real time.
- **CLI agents** (`vasp-agent.py` for workstations, `vasp-agent-slurm.py` for SLURM clusters) — read a plain-text `instructions.txt` file and generate all INCAR, KPOINTS, POSCAR, POTCAR, and job scripts automatically.

Both interfaces share the same `modules/` back-end.

1.1 Key features

- Functionals: PBEsol, PBE, R2SCAN, HSE06, RVV10, LDA, AM05
- Spin-orbit coupling (SOC) and collinear spin (ISPIN=2)
- Magnetic materials: correct MAGMOM per atom; spin-resolved band and DOS plots (spin-up / spin-down)
- GGA+U (Dudarev, LDAUTYPE=2) per element and orbital
- Automatic k-path detection via spglib (Setyawan–Curtarolo standard paths), with a geometric fallback
- Convergence testing: ENCUT and k-mesh sweeps
- DOS with orbital projections via `sumo`
- Phonons via phonopy, Wannier90 interface, DFPT Born charges
- LOBSTER bonding analysis (COHP/COBI/COOP) via the `08_lobster` step
- Smart KPAR/NCORE auto-defaults scaled with MPI task count
- SLURM workflow with dependency-chained job submission

1.2 Requirements

Package	Notes
VASP 5.4+	<code>vasp_std</code> , <code>vasp_nc1</code> (SOC), optionally <code>vasp_gam</code>
POTCAR library	PAW pseudopotentials in per-element folders
Python 3.8+	Conda environment recommended
Flask	<code>pip install flask</code> — GUI only
<code>sumo</code> ≥ 2.0	<code>pip install sumo</code> — band and DOS plots
MPI	<code>mpirun</code> , <code>mpiexec</code> , or <code>srun</code>
phonopy	optional; <code>conda install -c conda-forge phonopy</code>
wannier90.x	optional; <code>conda install -c conda-forge wannier90</code>

2 Installation

```
# 1. Clone the repository
git clone https://github.com/KeivanS/VASP-Flow.git
cd VASP-Flow

# 2. Install Python dependencies
pip install flask sumo

# 3. Optional: phonons and Wannier support
conda install -c conda-forge phonopy wannier90

# 4. First-time configuration (creates site.env from template)
make setup
nano site.env      # set VASP paths, POTCAR directory, MPI command
```

2.1 site.env — platform configuration

```
PYTHON          = python3
VASP_STD         = /path/to/vasp_std
VASP_NCL        = /path/to/vasp_ncl
VASP_GAM        = /path/to/vasp_gam      # optional
VASP_POTCAR_DIR = /path/to/potpaw_PBE
MPI_LAUNCH      = mpirun -np            # or: srun
MPI_NP          = 16
WANNIER90_X     = wannier90.x
```

3 Browser GUI

3.1 Starting the GUI

```
make run      # launches on http://localhost:5001
```

On a SLURM cluster — run on the login node, open on your laptop via SSH port-forwarding:

```
# On your laptop:
ssh -L 5001:localhost:5001 username@cluster

# On the cluster login node:
make run

# Open http://localhost:5001 in your laptop's browser
```

3.2 Three-tab workflow

1. **Setup** — configure the project, then click *Generate Workflow*.
2. **Workflow** — run each calculation step; live VASP output streams to the browser.
3. **Results** — band-structure and DOS plots; OUTCAR analysis.

To return to a previous project use the *Select project* dropdown. → **Workflow** jumps to the run tab; **Edit & Regenerate** re-opens Setup with all settings pre-filled.

3.3 Setup tab — field guide

3.3.1 Structure and POTCAR

Field	Description
Project name	Folder created in the working directory.
POSCAR	Paste the crystal structure, or click <i>Load file</i> .
POTCAR directory	Path to your PAW library. If an element has multiple variants (Ga vs Ga_d) a chooser appears.

3.3.2 Functional

Choice	Description
PBEsol	Optimised for solids; recommended starting point.
PBE	Standard GGA; widely used.
R2SCAN	Meta-GGA; good accuracy/cost balance.
HSE06	Hybrid — accurate band gaps, significantly slower.
RVV10	van der Waals; for layered or molecular materials.
LDA / AM05	Rarely needed.

3.3.3 Spin / SOC

Choice	Effect
None	Non-magnetic.
Collinear spin (ISPIN=2)	Sets ISPIN = 2; reveals the <i>Magnetic moment / atom</i> field (Section 7.3).
SOC z/x/y	Sets LSORBIT, LNONCOLLINEAR, SAXIS; uses the vasp_nc1 binary.

3.3.4 Other flags

Field	Effect
Hexagonal BZ	Correct <i>M</i> point for hexagonal structures.
2D / slab	Sets $k_z = 1$ (Gamma-only out-of-plane).
GGA+U	Adds Hubbard <i>U</i> rows per element/orbital (Section 7.4).
Execution profile	Select <i>SLURM HPC</i> to generate <i>sbatch</i> scripts; edit <i>profiles/slurm.json</i> first.
MPI tasks	CPU cores per calculation.

3.4 Task checkboxes

Task	Directory	Notes
Structure relaxation	01_relax	IBRION=2; run first if geometry is not pre-optimised.
SCF	02_scf	Required before bands or DOS.
Band structure	03_bands	Line-mode KPOINTS; Section 9.
DOS	04_dos	Dense k-mesh; orbital projections via sumo.
DFPT	06_dfpt	Born charges + dielectric (IBRION=8).
Phonons	07_phonons	Phonopy finite-displacement; DFPT needed for NAC.
Wannier90	05_wannier	NSCF interface; Section 14.
LOBSTER	08_lobster	COHP/COBI/COOP bonding analysis; needs SCF; Section 16.

Note. Normal execution order: **relaxation** → **SCF** → **bands** → **DOS**. Each step copies POSCAR/CHGCAR from the previous one automatically.

3.5 Band structure options

Leave the *k-path* field blank to trigger **automatic detection** from the POSCAR lattice vectors (Section 9). Enter a path manually to override, e.g. **G-M-K-G**.

3.6 DOS options

Add projection rows to request orbital-resolved DOS:

Element: **Mo** Orbitals: **d** Element: **S** Orbitals: **p**

For spin-polarised calculations sumo plots spin-up positive and spin-down mirrored below zero automatically.

3.7 Workflow tab

Each step shows three controls: **Run** (submit the step), **Edit files** (open INCAR/KPOINTS/POSCAR in the browser; a **.bak** is created on first edit), and a **status badge**. **Run All Steps** chains every step and stops on failure.

3.8 Results tab

Panel	Contents
Band structure	Spin-polarised: blue = $\uparrow$, red dashed = $\downarrow$, same k-axis, $E_F = 0$. Non-magnetic: single-colour sumo plot. If sumo fails (e.g. a pymatgen gap-detection crash) the GUI falls back to a built-in vasprun.xml plotter so a band plot still renders.
Cumulative DOS	Stacked filled-area by element. Spin-polarised: up positive, down mirrored.
Projected DOS	Orbital-resolved via sumo; spin-down negated automatically.
OUTCAR analysis	Energy, Fermi level, magnetic moment, forces.

4 CLI Agents

4.1 vasp-agent.py — workstation

```
./vasp-agent.py
./vasp-agent.py -i my_inst.txt -s struct.vasp

cd ProjectName
./run_all.sh           # runs every step in sequence
./analyze.sh           # plots after all steps complete
```

4.2 vasp-agent-slurm.py — SLURM clusters

```
./vasp-agent-slurm.py           # uses profiles/slurm.json
./vasp-agent-slurm.py --profile slurm

cd ProjectName
./submit_all.sh               # sbatch with dependency chaining
```

```
squeue -u $USER          # monitor jobs
./analyze.sh             # run locally after jobs finish
```

Two-phase workflow when convergence tests are requested:

```
./submit_convergence.sh  # STEP 1 -- submit convergence jobs
# review results, then:
./submit_calculations.sh # STEP 2 -- prompts ENCUT/k-mesh, submits
```

4.3 Generated directory structure

```
ProjectName/
  instructions.txt
  POSCAR
  POTCAR                built once from $VASP_POTCAR_DIR
  01_relax/             INCAR  KPOINTS  POSCAR  POTCAR  run.sh
  02_scf/
  03_bands/
  04_dos/
  08_lobster/           if LOBSTER task selected (ISYM=0 NSCF +
    lobster)
  analysis/             plots generated by analyze.sh
  run_all.sh            workstation sequential runner
  submit_all.sh         SLURM job submission (dependency chained)
  analyze.sh            post-processing: plots + OUTCAR extraction
```

5 High-Throughput Screening (Materials Project)

`ht-mp-scf.py` drives a batch of SCF + ELF calculations straight from Materials Project IDs. For every `mp-id` it downloads the **primitive** cell from MP, stages a per-material `instructions.txt` (an SCF task plus an INCAR `scf: block setting LELF = .TRUE.` for the electron localization function), and writes a `runall.sh` that calls the chosen agent to build the inputs and then runs or submits each job. Each SCF uses a medium 10 10 10 Γ -mesh by default; edit the `KMESH` line in `_ht_inputs/<id>/instructions.txt` (or pass `--kmesh`) to change it.

```
pip install pymatgen mp-api          # MP download + primitive
standardisation
export MP_API_KEY=...                 # Materials Project API key
export VASP_POTCAR_DIR=...           # POTCAR library (used by the
agent)

# one mp-id per line (blank lines and # comments ignored)
printf 'mp-149\nmp-2534\n' > highthrouput_list

./ht-mp-scf.py --agent local --mpi 16      # workstation
./ht-mp-scf.py --agent slurm --profile slurm # cluster (chained by
default)

./runall.sh                            # build inputs + run/submit
every job
```

The ELFCAR for each material is written to `<mp-id>/02_scf/`. A summary `_ht_inputs/elements_Z.txt` tabulates every element in the screening set with its atomic number Z (from a built-in table — no MP query), plus a per-material breakdown.

Option	Meaning
<code>--agent</code>	<code>local</code> (vasp-agent.py) or <code>slurm</code> (vasp-agent-slurm.py); prompts if omitted.
<code>--list</code>	ID file; default <code>highthrouput_list</code> .
<code>--functional</code>	PBE (default), PBEsol, R2SCAN, ...
<code>--mpi</code>	MPI tasks per job.
<code>--encut</code>	Optional ENCUT override (eV).
<code>--kmesh</code>	SCF Γ -mesh, e.g. '8 8 8' or '12' (cubic); default medium 10 10 10. Per-material edits via the KMESH line in <code>instructions.txt</code> .
<code>--chain / --no-chain</code>	SLURM only; see below.

5.1 Execution order

Mode	Behaviour
<code>local</code>	<code>run_all.sh</code> blocks — materials run strictly one-at-a-time.
<code>slurm --chain</code> (default)	Each material's SCF is submitted with <code>--dependency=afterok</code> on the previous one, so the cluster runs them sequentially even though all are queued up front.
<code>slurm --no-chain</code>	Materials are submitted independently and run whenever the scheduler allows.

Note. The driver fetches structures over the network (Materials Project), so run it on a node with internet — typically the login node. `runall.sh` performs no downloads and can run on a compute node.

6 instructions.txt Format

6.1 Example

```
Project: My material name

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Fe-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Fe-d and O-p)

Convergence: Test k-points 6x6x6 to 12x12x12
             ENCUT 400-600 eV

MAGMOM: 4.0
NSW: 100
EDIFFG: -0.01
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 12:00:00
```

The parser is case-insensitive and searches the entire file, so section order is flexible.

6.2 Keyword reference

Keyword	Meaning
Project:	Project name; becomes the output folder.
MAGMOM:	Initial moment in μ_B/atom (collinear only). Written as MAGMOM = N*value . Default 2.0.
ISIF:	Relaxation type: 3=full, 2=ions only, 4=fixed volume.
NSW:	Max ionic steps. Default 100.
EDIFFG:	Force criterion in eV/Å. Default -0.01 .
ENCUT:	Manual plane-wave cutoff in eV.
KMESH:	Explicit Gamma k-mesh overriding the automatic density, e.g. 8 8 8 or 12 (cubic). Applies to every automatic mesh (relax/SCF/DOS); for 2-D give the third value (12 12 1).
PRESSURE:	External pressure for constant- P relaxation, e.g. 10 GPa or 50 kbar. Forces ISIF=3, IBRION=2, PSTRESS (Section 8.2).
INCAR: ... END_INCAR	Block of literal INCAR tags merged into the generated INCAR(s); optionally per-step (Section 8.1).
NKPTS:	k-points per band segment. Default 40.
MPI:	Total MPI tasks (nodes $\times$ cores/node).
KPAR:	Global k-point parallelisation.
NCORE:	Global cores-per-orbital.
SCF_KPAR: etc.	Per-step KPAR: also BANDS_, DOS_, RELAX_, DFPT_, PHONONS_.
SCF_NCORE: etc.	Per-step NCORE; same prefixes.
NODES:	Compute nodes (SLURM).
NTASKS_PER_NODE:	MPI tasks per node (SLURM).
PARTITION:	SLURM partition.
WALLTIME:	Time limit, e.g. 12:00:00.
ACCOUNT:	SLURM account / allocation.
DFPT_EDIFF:	EDIFF for DFPT. Default 1E-8.
PHONONS_DIM:	Supercell, e.g. 2 2 2.
PHONONS_MESH:	q-mesh, e.g. 20 20 20.
PHONONS_DISP:	Displacement in Å. Default 0.01.
PHONONS_BAND:	High-symmetry path for phonon bands.
PHONONS_NAC:	FALSE to disable NAC.
WANNIER_NUM_WANN:	Number of Wannier functions.
WANNIER_PROJ:	Projections, e.g. Ga:s;As:p.
WANNIER_EWIN:	Energy window, e.g. -5.0 to 15.0.

7 Methods

7.1 Functionals

```

Methods: PBEsol functional      # recommended for solids
Methods: R2SCAN functional      # meta-GGA
Methods: HSE06 functional       # hybrid -- more accurate but slower
Methods: RVV10 functional       # van der Waals

```

Default ENCUT values: PBE 400 eV, PBEsol 450 eV, R2SCAN 680 eV, HSE06 400 eV, RVV10 450 eV.

7.2 Spin-orbit coupling

Methods: SOC with magnetization in z-direction

Uses `vasp_ncl`. Sets `LSORBIT`, `LNONCOLLINEAR`, and `SAXIS`.

7.3 Magnetic materials (collinear spin)

Methods: collinear magnetization
MAGMOM: 4.0

Sets `ISPIN` = 2 and `MAGMOM` = $N * \text{value}$ in every `INCAR`, where N is the atom count from the `POSCAR`.

Tip. Typical starting moments: Fe, Co $\approx 4\text{--}5 \mu_B$; Ni $\approx 2 \mu_B$; Cr, Mn $\approx 3\text{--}4 \mu_B$; rare earths $\approx 7 \mu_B$. VASP converges the self-consistent value automatically.

Antiferromagnets — after generation, edit `INCAR` to alternate signs:

MAGMOM = 4.0 -4.0 4.0 -4.0 ! alternating per sublattice

Spin-resolved plots produced automatically after `DOS/bands` run:

- *Cumulative DOS* — spin-up above the x-axis; spin-down mirrored below zero; elements colour-coded; solid = $\uparrow$, dashed = $\downarrow$.
- *Band structure* — spin-up in blue, spin-down in red dashed, same k-axis, $E_F = 0$.
- *Projected DOS* — sumo natively negates the spin-down channel in the orbital-resolved PDF.

Check the converged moment:

```
grep "magnetization_␣(x)" OUTCAR | tail -5
```

7.4 GGA+U

Methods: GGA+U with U=4.0 on Fe-d orbitals
GGA+U with U=3.0 on Ni-d orbitals
GGA+U with U=6.0 on Ce-f orbitals

Dudarev scheme (`LDAUTYPE=2`); $J = 0$ is set automatically. Combine freely with **collinear magnetization**.

8 Advanced INCAR Control

Two mechanisms let you go beyond the built-in keywords without editing each generated `INCAR` by hand: **raw INCAR passthrough** (inject any VASP tag directly) and **constant-pressure relaxation**.

8.1 Raw INCAR passthrough

Place literal `INCAR` lines — in ordinary VASP syntax — inside an `INCAR: ... END_INCAR` block in your `instructions.txt`. They are merged into the generated `INCAR(s)`:

```
INCAR:                                # global block -- applies to every step
    LREAL    = Auto
    ALGO      = Fast
```

```
NELMIN = 6
END_INCAR
```

A tag that the agent already writes is **overwritten in place** (so `ALGO = Fast` above replaces the default `ALGO = Normal`); any tag the agent does not normally emit is appended under a clearly labelled `# User INCAR overrides` section, so nothing is silently lost.

To target a single step, name it on the header line. Recognised steps are `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`:

```
INCAR scf:                # applies only to 02_scf
    EDIFF = 1E-8
    SIGMA = 0.10
END_INCAR
```

Note. Precedence: a *per-step* block beats the *global* block, and both beat the agent's generated defaults. This is the recommended way to set tags the parser has no dedicated keyword for (e.g. `LMAXMIX`, `ADDGRID`, `LVTOT`, `IVDW`, `AMIX`).

8.2 Constant-pressure relaxation

A single `PRESSURE` line turns the relaxation into a constant-pressure optimisation: the agent forces `ISIF = 3` (cell shape + volume + ions) and `IBRION = 2`, and imposes the target pressure through `PSTRESS`.

```
PRESSURE: 10 GPa          # also accepted: "5 kbar", "constant
    pressure 8"
```

VASP's `PSTRESS` tag is in **kBar** (1 GPa = 10 kBar). The agent converts automatically; if no unit is given, **GPa is assumed**. The resulting `01_relax/INCAR` reads, e.g.:

```
# Constant-pressure relaxation @ 10 GPa (PSTRESS = 100 kBar)
IBRION = 2
ISIF   = 3
NSW    = 100
EDIFFG = -0.01
PSTRESS = 100 ! external pressure in kBar (10 GPa)
```

Tip. `PSTRESS` adds the *PV* term so the relaxed cell sits at the requested external pressure. Use a tight `EDIFFG` and a well-converged `ENCUT`; the Pulay stress error grows at high pressure, so verify with a higher `ENCUT` before trusting the equilibrium volume.

9 K-Path for Band Structure

9.1 Auto-detection (recommended)

Omit the path in `instructions.txt` or leave the field blank in the GUI:

```
Tasks: band structure      # no "along ..." -> auto-detect from
    POSCAR
```

Since v1.6 the generator first determines the *actual space group* of the POSCAR with `spglib` (via `pymatgen's KPathSetyawanCurtarolo`) and uses the standard Setyawan–Curtarolo high-symmetry path for that space group. Every high-symmetry point is transformed through Cartesian reciprocal space into the reciprocal basis of *this* POSCAR, so the path is correct for the actual cell used in the SCF regardless of orientation or setting.

If `spglib` or `pymatgen` is not installed (or the symmetry analysis fails), the generator falls back to the geometric classifier below: it reads the POSCAR lattice vectors, computes cell lengths

and angles, and selects a default path by Bravais type. 2-D structures always use the geometric route so the in-plane path is preserved.

Type	Detection criterion	Default path
cF (FCC)	$a=b=c$, angles $\approx 60^\circ$	G-X-W-K-G-L-U-W
cI (BCC)	$a=b=c$, angles $\approx 109.5^\circ$	G-H-N-G-P-H
cP (cubic)	$a=b=c$, 90°	G-X-M-G-R-X
hP (hexagonal)	$a\approx b$, $\gamma=120^\circ$	G-M-K-G-A-L-H-A
tP (tetragonal)	$a\approx b\neq c$, 90°	G-X-M-G-Z-R-A-Z
oP (orthorhombic)	90° , $a\neq b\neq c$	G-X-S-Y-G-Z-U-R-T-Z
rP (rhombohedral)	$a=b=c$, equal angles	G-T-F-G-L
mP (monoclinic)	$\alpha=\gamma=90^\circ$, $\beta\neq 90^\circ$	G-Y-A-Z-G-B-D

Note. For **conventional** FCC or BCC cells (right angles, 4 or 2 atoms), auto-detection returns the cubic (cP) path. Use the **primitive** cell for an unfolded FCC/BCC band structure — the primitive FCC cell has angles $\approx 60^\circ$ and is recognised automatically.

The detection route appears in the KPOINTS comment:

```
Line-mode KPOINTS [spglib ... -- auto-detected] # spglib route
Line-mode KPOINTS [FCC -- auto-detected]       # geometric
fallback
```

9.2 Manual override

Tasks: band structure along G-X-W-K-G-L

Label	Coords	Use	
G	(0, 0, 0)	all	Γ
M	(0.5, 0.5, 0)	cubic, tet	(0.5, 0, 0) in hexagonal
K	(1/3, 1/3, 0)	hexagonal	
R	(0.5, 0.5, 0.5)	cubic	
Z	(0, 0, 0.5)	general	
A	(0, 0, 0.5)	hex, tet	
L	(0.5, 0, 0.5)	hexagonal	
H	(1/3, 1/3, 0.5)	hexagonal	
S	(0.5, 0.5, 0)	orthorhombic	
W	(0.5, 0.25, 0.75)	FCC primitive	
P	(0.25, 0.25, 0.25)	BCC primitive	
N	(0, 0, 0.5)	BCC primitive	

10 DOS Projections

All of the following forms are accepted:

```
DOS (projected on Mo-d and S-p)
DOS with projections on Ni:d and Ni:s
DOS with projection on Fe-d and O-p
```

Orbital channels: s, p, d, f. Separator is either - or :. Sumo generates an orbital-resolved PDF in analysis/.

11 Convergence Tests

```
# Range (auto-steps)
Convergence: Test k-points from 6x6x1 to 18x18x1      # 2D material
               Test k-points from 6x6x6 to 12x12x12    # 3D material
               ENCUT 400-600 eV
```

```
# Explicit mesh list
Convergence: Test k-points 6x6x3, 12x12x6, 18x18x9
```

Results in 00_convergence/kpoints/ and 00_convergence/encut/.

```
./run_convergence.sh          # workstation
./submit_convergence.sh       # SLURM
./analyze_convergence.sh      # plots: energy / pressure / forces
```

12 Parallelization

12.1 Auto-defaults

When KPAR/NCORE are not specified the agent computes from total tasks N :

Step	KPAR	NCORE	Notes
SCF / DOS / relax	$\lfloor \sqrt{N} \rfloor$	N/KPAR	
bands	1	N	Required for line-mode
DFPT	1	N	Required for IBRION=8
phonons	1	$\lfloor \sqrt{N} \rfloor$	

Note. $\text{KPAR} \times \text{NCORE}$ must divide N evenly. The agent rounds down automatically to the nearest valid value.

12.2 Manual overrides

```
MPI: 128          KPAR: 4          NCORE: 8

# Per-step (take priority over globals)
SCF_KPAR: 4        BANDS_KPAR: 1      DOS_KPAR: 8
SCF_NCORE: 8        BANDS_NCORE: 16    DOS_NCORE: 8
RELAX_KPAR: 2        DFPT_KPAR: 1      PHONONS_KPAR: 1
```

13 SLURM Cluster Settings

13.1 profiles/slurm.json

```
{
  "vasp_std": "vasp_std",  "vasp_ncl": "vasp_ncl",
  "mpi_cmd":  "srun",
  "modules":  ["intel/2021", "impi/2021", "vasp/6.3.2"],
  "slurm": {
    "partition": "standard",
    "nodes":     2,
    "ntasks_per_node": 64,
    "time":      "12:00:00",
    "account":   "mygroup"
  }
}
```

13.2 Per-project overrides (in instructions.txt)

```
NODES: 4      NTASKS_PER_NODE: 32      PARTITION: gpu
WALLTIME: 48:00:00      ACCOUNT: myallocation
```

14 Wannier90

```
Tasks: Wannier90 wannierization

WANNIER_NUM_WANN: 8
WANNIER_PROJ: Ga:s;As:p
WANNIER_EWIN: -5.0 to 15.0
```

The agent generates `05_wannier/wannier90.win` with the NSCF k-mesh and projection block. After the NSCF run, execute `wannier90.x` from the `05_wannier/` directory.

15 Phonons (phonopy)

```
Tasks: phonons lattice dynamics

PHONONS_DIM: 2 2 2
PHONONS_MESH: 20 20 20
PHONONS_DISP: 0.01
PHONONS_BAND: G M K G A
PHONONS_NAC: FALSE      # disable non-analytic correction
```

Note. For the non-analytic correction (important for polar materials), include DFPT in your task list so Born effective charges are computed.

16 LOBSTER Bonding Analysis (COHP / COBI / COOP)

Adding LOBSTER COHP/COBI analysis (keywords: `lobster`, `cohp`, `cobi`, `coop`, `bonding`) to the task list creates `08_lobster`: a symmetry-off NSCF that reads `02_scf/CHGCAR`, followed by a run of the `lobster` binary.

- **ISYM=0 is required** — a symmetrised k-mesh gives unusable COHP/ICOHP values (wrong sign, errors above 100%). The generated INCAR sets it automatically. With SOC enabled, **ISYM=-1** is used instead: spin-orbit coupling breaks time-reversal symmetry, which ISYM=0 would still assume.
- NBANDS is auto-set to at least the LOBSTER basis-function count; LMAXMIX is chosen from the elements (6 for *f*, 4 for *d*, 2 for *sp*).
- The LOBSTER binary is set with `LOBSTER_X` in `site.env` (default `lobster-5.1.0-OSX`); it can be overridden at runtime via `$LOBSTER_BIN`.

```
Project: NaCl
Methods: PBE functional
Tasks: SCF, LOBSTER COHP/COBI analysis
MPI: 16
```

Run with `./vasp-agent.py` (builds and runs `02_scf + 08_lobster`); the GUI shows COHP/COBI/COOP plots on the Results tab. To aggregate averaged COHP/COBI over many projects into `lobster_acohp_acobi.c` run `python3 lobster_postprocess.py`.

17 Worked Examples

17.1 MoS₂ monolayer with SOC

```
Project: MoS2 monolayer

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks:  structure relaxation
        SCF calculation
        band structure
        DOS (projected on Mo-d and S-p)

Convergence: Test k-points 8x8 to 20x20
             ENCUT 400-600 eV

MPI: 64
```

The generator detects hexagonal (hP) and uses G-M-K-G-A-L-H-A.

17.2 Nickel: magnetism + GGA+U

```
Project: Nickel GGA+U

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Ni-d orbitals

Tasks:  structure relaxation
        SCF calculation
        band structure
        DOS (projected on Ni-d and Ni-s)

MAGMOM: 2.0
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 08:00:00
```

With a primitive FCC POSCAR (angles $\approx 60^\circ$) the generator detects cF and uses G-X-W-K-G-L-U-W. Band and DOS plots show both spin channels.

17.3 GaAs: phonons with NAC

```
Project: GaAs phonons

Methods: PBEsol functional

Tasks:  SCF calculation
        DFPT Born charges and dielectric
        phonons lattice dynamics

PHONONS_DIM: 4 4 4
PHONONS_MESH: 20 20 20
PHONONS_BAND: G X W L G K
```

```
MPI: 64      WALLTIME: 24:00:00
```

17.4 High-pressure relaxation with custom INCAR tags

A constant-pressure cell relaxation at 10 GPa, combined with raw INCAR passthrough to tighten convergence and add tags the parser has no dedicated keyword for:

```
Project: Silicon 10GPa

Methods: PBEsol functional

Tasks: structure relaxation
       SCF calculation
       band structure

PRESSURE: 10 GPa          # constant-P relax -> ISIF=3, IBRION=2,
PSTRESS=100

# Tags applied to every step
INCAR:
  LREAL  = .FALSE.
  ADDGRID = .TRUE.
  LMAXMIX = 4
END_INCAR

# Tighter electronic convergence for the relaxation only
INCAR relax:
  EDIFF  = 1E-8
  EDIFFG = -0.005
END_INCAR

ENCUT: 600
MPI: 64
```

The 01_relax/INCAR is generated with ISIF=3, PSTRESS=100, the global tags appended, and EDIFF/EDIFFG overridden to the per-step values; 02_scf and 03_bands receive only the global block.

17.5 Bi₂Se₃: topological insulator

```
Project: Bi2Se3 topological

Methods: PBEsol functional
        SOC with magnetization in z-direction

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Bi-p and Se-p)
       Wannier90 wannierization

Convergence: Test k-points from 8x8x4 to 16x16x8
             ENCUT 400-600 eV

WANNIER_NUM_WANN: 18
WANNIER_PROJ: Bi:p;Se:p
WANNIER_EWIN: -2.0 to 2.0
```


MPI: 128

18 Common OUTCAR Checks

```

grep "reached_required_accuracy" OUTCAR # converged?
grep "energy_without" OUTCAR | tail -1 # total energy
grep "E-fermi" OUTCAR | tail -1 # Fermi level
grep "magnetization(x)" OUTCAR | tail -5 # moment
grep "TOTAL-FORCE" OUTCAR | tail -20
grep -A3 "BORN_EFFECTIVE_CHARGES" OUTCAR | head -20

```

19 Troubleshooting

Symptom	Solution
POTCAR not found	Check VASP_POTCAR_DIR in <code>site.env</code> or GUI System Setup. Use the POTCAR chooser for unusual variants (Ga_d, Fe_sv).
vasp_std not found	Set the full binary path in <code>site.env</code> or GUI System Setup.
WARNING: may not have converged	Increase NSW (ionic) or NELM (electronic) in INCAR and re-run.
Wrong bands ($W = \Gamma$)	Conventional cubic cell detected; use the <i>primitive</i> cell for FCC/BCC so auto-detection picks cF/cI.
Projected DOS missing	Check <code>instructions.txt</code> : <code>projected on</code> and <code>projections on</code> are both accepted. Verify element names match POSCAR.
KPAR error	$KPAR \times NCORE$ must divide total MPI tasks. Use auto-defaults or set explicitly valid values.
GUI unreachable	Check SSH tunnel: <code>ssh -L 5001:localhost:5001 user@cluster</code> .
Project missing from dropdown	Folder must contain POSCAR + <code>settings.json</code> in the directory where <code>vasp-gui.py</code> was started.
Band plot empty or not shown	Usually <code>sumo-bandplot</code> crashed on this band structure; the GUI now falls back to a built-in plotter automatically. An empty plot also means a wrong E_F : the Fermi level is read from 04_dos/OUTCAR (then 02_scf/OUTCAR), since 03_bands eigenvalues share the NSCF/DOS energy reference.

20 Version History

- v1.0 (2026-03-23)** Initial release — GUI, workstation agent, PBEsol/R2SCAN/HSE06, SOC, GGA+U, convergence tests, Wannier90, phonopy.
- v1.1 (2026-04-05)** Per-task KPAR/NCORE smart defaults; SLURM agent with dependency-chained submission; cumulative DOS by element (no pymatgen); copy-from-relax/scf scripts.
- v1.2 (2026-06)** Collinear magnetism: MAGMOM per atom, spin-resolved band and DOS plots. Auto k-path detection (8 crystal systems). Fixed DOS projection parsing. Separate GUI and agent quick-reference sheets. Rewritten comprehensive LaTeX manual.

- v1.3 (2026-06-15)** Raw INCAR passthrough (INCAR: ... END_INCAR, global or per-step) for arbitrary VASP tags. Constant-pressure relaxation via PRESSURE (ISIF=3, IBRION=2, PSTRESS).
- v1.4 (2026-06-19)** High-throughput driver `ht-mp-scf.py`: Materials Project primitive cells → batch SCF + ELF, with per-material element/*Z* table and cross-material `afterok` chaining on SLURM.
- v1.5 (2026-06-24)** Robust band plotting across the GUI and both CLI agents (workstation + SLURM): the built-in `vasprun.xml` plotter now handles non-spin as well as spin, and is used automatically when `sumo-bandplot` crashes or is unavailable. Fixed the matplotlib fallback (`plot_results.py`) to resolve the real Fermi level and emit PNG, not just PDF.
- v1.6 (2026-07-08)** Band k-path derived from the real space group via `spglib` (Setyawan–Curtarolo) in all agents and the GUI, with the geometric classifier kept as fallback. New `08_lobster` step: LOBSTER COHP/COBI/COOP bonding analysis with GUI plots and the `lobster_postprocess.py` aggregator; the NSCF uses ISYM=0, or ISYM=-1 when SOC is on. Full GPL-3.0 text in LICENSE (GitHub now detects the license correctly). Manual and quick-reference sheets brought up to date.

Addendum: High-Throughput Quick Reference

Cheat-sheet for `ht-mp-scf.py` (full description in Section 5). For each Materials Project ID it downloads the **primitive** cell, stages an SCF calculation that also writes the electron localization function (LELF = .TRUE. → ELFCAR) on a medium 10 10 10 Γ -mesh (default; editable per material or via `--kmesh`), and builds a `runall.sh`.

```

pip install pymatgen mp-api
export MP_API_KEY=...           # Materials Project key
export VASP_POTCAR_DIR=...      # POTCAR library (used by the agent)

printf 'mp-149\nmp-2534\n' > highthrouput_list  # one mp-id per line

./ht-mp-scf.py --agent local    --mpi 16         # workstation,
    sequential
./ht-mp-scf.py --agent slurm    --profile slurm   # cluster,
    afterok-chained
./ht-mp-scf.py --agent slurm    --no-chain       # cluster,
    independent

./runall.sh                     # build inputs + run/submit each SCF

```

Output	Location
ELFCAR (electron localization)	<mp-id>/02_scf/ELFCAR
Staged inputs (POSCAR + instructions)	_ht_inputs/<mp-id>/
Element / <i>Z</i> table	_ht_inputs/elements_Z.txt

Note. Transfer to a workstation: only the project folder is needed (code, modules/, profiles/); VASP binaries and the POTCAR library already live there. `ht-mp-scf.py` needs internet for the MP download, so run it on the login node — `runall.sh` does none.

Note. If the download step errors with 'dict' object has no attribute 'number': that is an `spglib` / `pymatgen` version mismatch. Fix the root cause with `pip install -U spglib`. The driver also falls back to `Structure.get_primitive_structure()` automatically, so a run is never blocked by it.

Source: <https://github.com/KeivanS/VASP-Flow> Compiled: July 8, 2026 Maintained by Keivan Esfarjani